

Object Oriented Programming In Java

Saurabh Pandey
Associate Professor
Sitare University

Java Operators

● Post and pre increment

Operator	Operation
++ --	increment, decrement
+ -	unary plus, minus
!	boolean not
(<type>)	cast to <type>
* / %	multiplication, division, remainder
+ -	addition/concatenation, subtraction
< <= > >=	relational ordering
== !=	relational equality, inequality
&&	boolean and
	boolean or
= += -= *= /= %=	assignments

Java Operators

Difference between (&&, ||) and (&,|)

&&, || : These are called Logical AND and Logical OR operator

&, | : These are called bitwise AND and bitwise OR operator

S.N.	Basis	& Operator	&& Operator
1	Operator	It is a bitwise AND operator.	It is a logical AND operator.
2	Evaluation	It evaluates both the left and right side of the given expression.	It only evaluates the left sides of the given expression.
3	Operates on	It operates on Boolean data types as well as on bits.	It operates only on Boolean datatype.
4	Uses	Used to check logical condition and also used to mask off certain bits such as parity bits.	Used only to check the logical conditions.
5	Example	<code>z = x & y</code>	<code>if (y > 1 && y > x)</code>

Conditional Statements

Java has the following conditional statements:

- Use if to specify a block of code to be executed, if a specified condition is true
- Use else to specify a block of code to be executed, if the same condition is false
- Use else if to specify a new condition to test, if the first condition is false
- Use switch to specify many alternative blocks of code to be executed

if syntax

else syntax

```
if (condition) {  
    // block of code to be executed if the condition is true  
}
```

else if syntax

```
if (condition) {  
    // block of code to be executed if the condition is true  
} else {  
    // block of code to be executed if the condition is false  
}  
  
if (condition1) {  
    // block of code to be executed if condition1 is true  
} else if (condition2) {  
    // block of code to be executed if the condition1 is false and condition2 is true  
} else {  
    // block of code to be executed if the condition1 is false and condition2 is false  
}
```

Conditional Statements

- Ternary operator(short hand if else) syntax

```
variable = (condition) ? expressionTrue : expressionFalse;
```

There could be nested if else statements also.

- Switch statement

```
switch(expression) {  
  case x:  
    // code block  
    break;  
  case y:  
    // code block  
    break;  
  default:  
    // code block  
}
```

break and continue keywords

You have already seen the break statement used in an earlier chapter of this tutorial. It was used to "jump out" of a switch statement.

The break statement can also be used to jump out of a loop.

```
for (int i = 0; i < 10; i++) {  
    if (i == 4) {  
        break;  
    }  
    System.out.println(i);  
}
```

The continue statement breaks one iteration (in the loop), if a specified condition is true, and continues with the next iteration in the loop.

```
for (int i = 0; i < 10; i++) {  
    if (i == 4) {  
        continue;  
    }  
    System.out.println(i);  
}
```

Loops in java

●While loop

```
while (condition) {  
    // code block to be executed  
}
```

●Do/While loop

```
do {  
    // code block to be executed  
}  
while (condition);
```

Loops in java

● For loop

```
for (statement 1; statement 2; statement 3) {  
    // code block to be executed  
}
```

Statement 1 is executed (one time) before the execution of the code block.

Statement 2 defines the condition for executing the code block.

Statement 3 is executed (every time) after the code block has been executed.

```
for (int i = 0; i < 5; i++) {  
    System.out.println(i);  
}
```

There could be nested loops in your code.

Loops in java

- For Each loop

It is used exclusively to loop through elements in an standard data structure as array or arraylist.

```
for (type variableName : arrayName) {  
    // code block to be executed  
}
```

```
String[] cars = {"Volvo", "BMW", "Ford", "Mazda"};  
for (String i : cars) {  
    System.out.println(i);  
}
```

Packages

- In the Java API, classes are grouped into packages. To use a class in the API, you have to know which package the class is in.

Packages are important for three main reasons.

- First, they help the overall organization of a project or library. Rather than just having one horrendously large pile of classes, they're all grouped into packages for specific kinds of functionality (like GUI, or data structures, or database stuff, etc.)
- Second, packages give you a name scoping, to help prevent collisions. If you and 12 other programmers in your company all decide to make a class with the same name. If you have a class named Set and someone else (including the Java API) has a class named Set, you need some way to tell the JVM which Set class you're trying to use.
- Third, packages provide a level of security, because you can restrict the code you write so that only other classes in the same package can access it.

import the package or type class with full package name

Object Equality

- When we compare primitives with `==` operator, it compares values of the primitives.
- When you compare two object references, it checks only reference values not the objects.
- If you want to check two objects equality, use `equals()` method which is method of `Object` class. `Object` class is mother of all classes in java.